CSC 340-03

Programming Methodology
Week 12

- Review Week 11

Instructor: Joseph Hui

Email: jhui@sfsu.edu

Basic Sorting Algorithms

- The Selection Sort
- The Bubble Sort
- The Insertion Sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

- Select the largest item
  - Place in last position
- Select the next largest item
  - Place in 2nd to last position
- Repeat for remaining unsorted items
  - Requires several passes over the data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

Gray elements are selected;
blue elements comprise the sorted portion of the array.

Initial array:	29	10	14	37	13
After 1st swap:	29	10	14	13	**37**
After 2nd swap:	13	10	14	**29**	**37**
After 3rd swap:	13	10	**14**	**29**	**37**
After 4th swap:	**10**	**13**	**14**	**29**	**37**

- To sort an array into ascending order, you first search for the largest item.
- Place largest item in the last position of the array
- Swap the last item with the largest item, even if these items happen to be identical.
- Repeat

FIGURE 11-1 A selection sort of an array of five integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

```
   for (int last = n - 1; last >= 1; last--)
   {
      // At this point, theArray[last+1..n-1] is sorted, and its
      // entries are greater than those in theArray[0..last].
      // Select the largest entry in theArray[0..last]
      int largest = findIndexofLargest(theArray, last+1);

      // Swap the largest entry, theArray[largest], with
      // theArray[last]
      std::swap(theArray[largest], theArray[last]);
   }  // end for
}  // end selectionSort

int findIndexofLargest(const ItemType theArray[], int size)
{
   int indexSoFar = 0;  // Index of largest entry found so far
   for (int currentIndex = 1; currentIndex < size; currentIndex++)
   {
      // At this point, theArray[indexSoFar] >= all entries in
      // theArray[0..currentIndex - 1]
      if (theArray[currentIndex] > theArray[indexSoFar])
         indexSoFar = currentIndex;
   }  // end for

   return indexSoFar;  // Index of largest entry
}  // end findIndexofLargest
```

Note this is a nested loop for n

LISTING 11-1 An implementation of the selection sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

- Analysis
  - Selection sort is $O(n^2)$
  - Appropriate for small *n*
  - $O(n^2)$ grows rapidly
- Could be a good choice when
  - Data moves are costly,
  - But comparisons are not
- Pros
  - Easy to understand
  - Minimal additional space required
- Cons
  - $O(n^2)$ – slow
  - Does not preserve relative order of items with equal keys (not stable)
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

- Compares adjacent items
  - Exchanges them if out of order
  - Requires several passes over the data
- When ordering successive pairs
  - Largest item bubbles to end of the array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

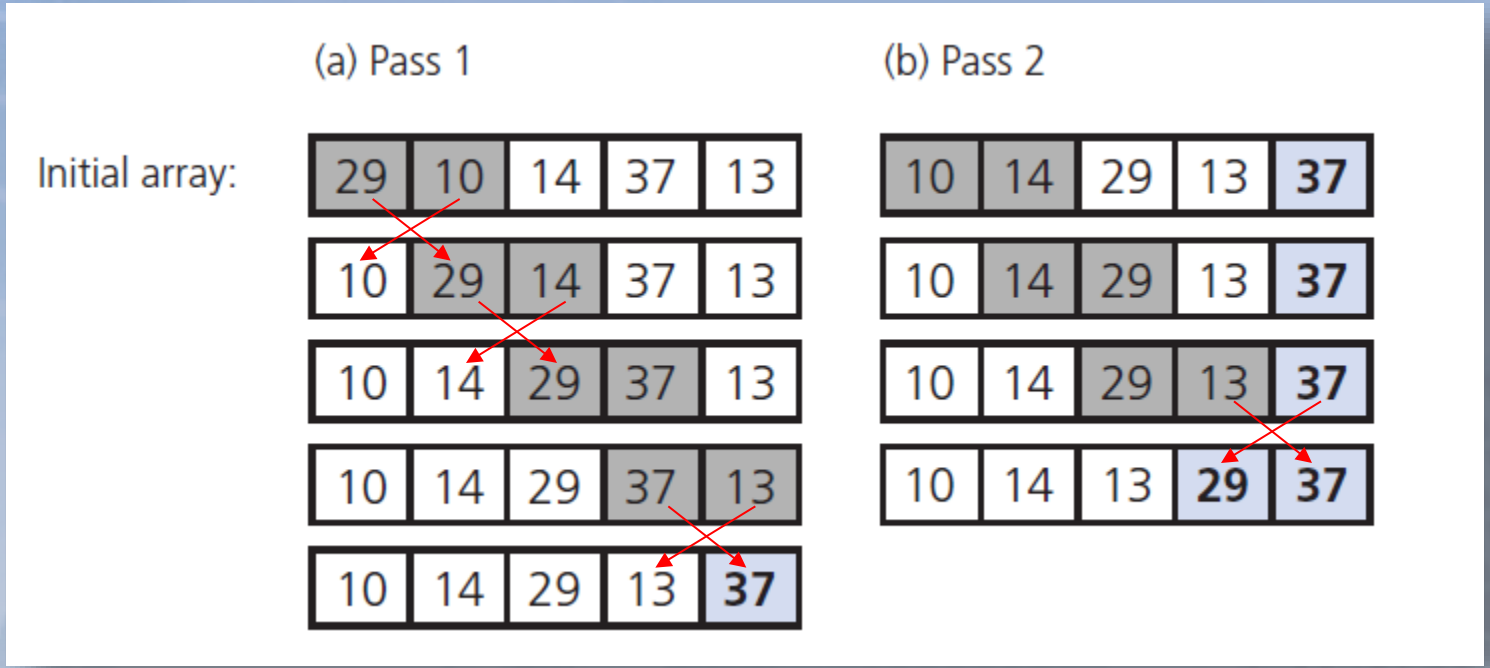


FIGURE 11-2 First two passes of a bubble sort of an array of five integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

LISTING 11-2 An implementation of the bubble sort

```
/** Sorts the items in an array into ascending order.
 @pre  None.
 @post  theArray is sorted into ascending order; n is unchanged.
 @param theArray  The given array.
 @param n  The size of theArray. */
void bubbleSort(ItemType theArray[], int n)
{
   bool sorted = false;     // False when swaps occur
   int pass = 1;
   while (!sorted && (pass < n))
   {
      // At this point, theArray[n+1-pass..n-1] is sorted
      // and all of its entries are > the entries in theArray[0..n-pass]
      sorted = true;        // Assume sorted
      for (int index = 0; index < n - pass; index++)
      {
         // At this point, all entries in theArray[0..index-1]
         // are <= theArray[index]
         int nextIndex = index + 1;
         if (theArray[index] > theArray[nextIndex])
         {
            // Exchange entries
            std::swap(theArray[index], theArray[nextIndex]);
            sorted = false;  // Signal exchange
         }  // end if
      }  // end for
      // Assertion: theArray[0..n-pass-1] < theArray[n-pass]

      pass++;
   }  // end while
}  // end bubbleSort
```

Note this is a nested loop for n

LISTING 11-2 An implementation of the bubble sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

- Analysis
  - Worst case $O(n^2)$
  - Best case (array already in order) is $O(n)$
- Pros
  - Easy to understand
  - No additional space required
  - Preserves relative order of items with equal keys (stable)
- Cons
  - $O(n^2)$ – slow
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

- Take each item from unsorted region
  - Insert it into correct order in sorted region

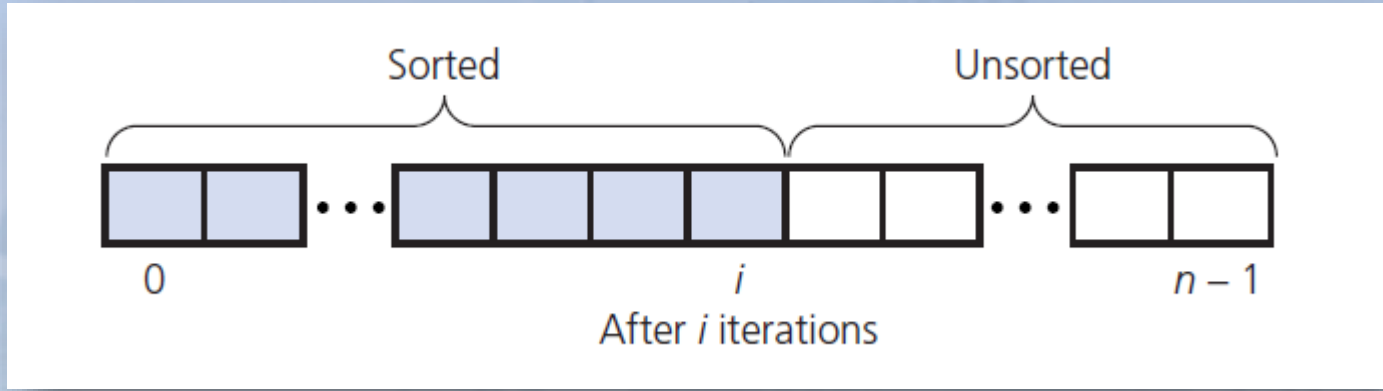


FIGURE 11-3 An insertion sort partitions the array into two regions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

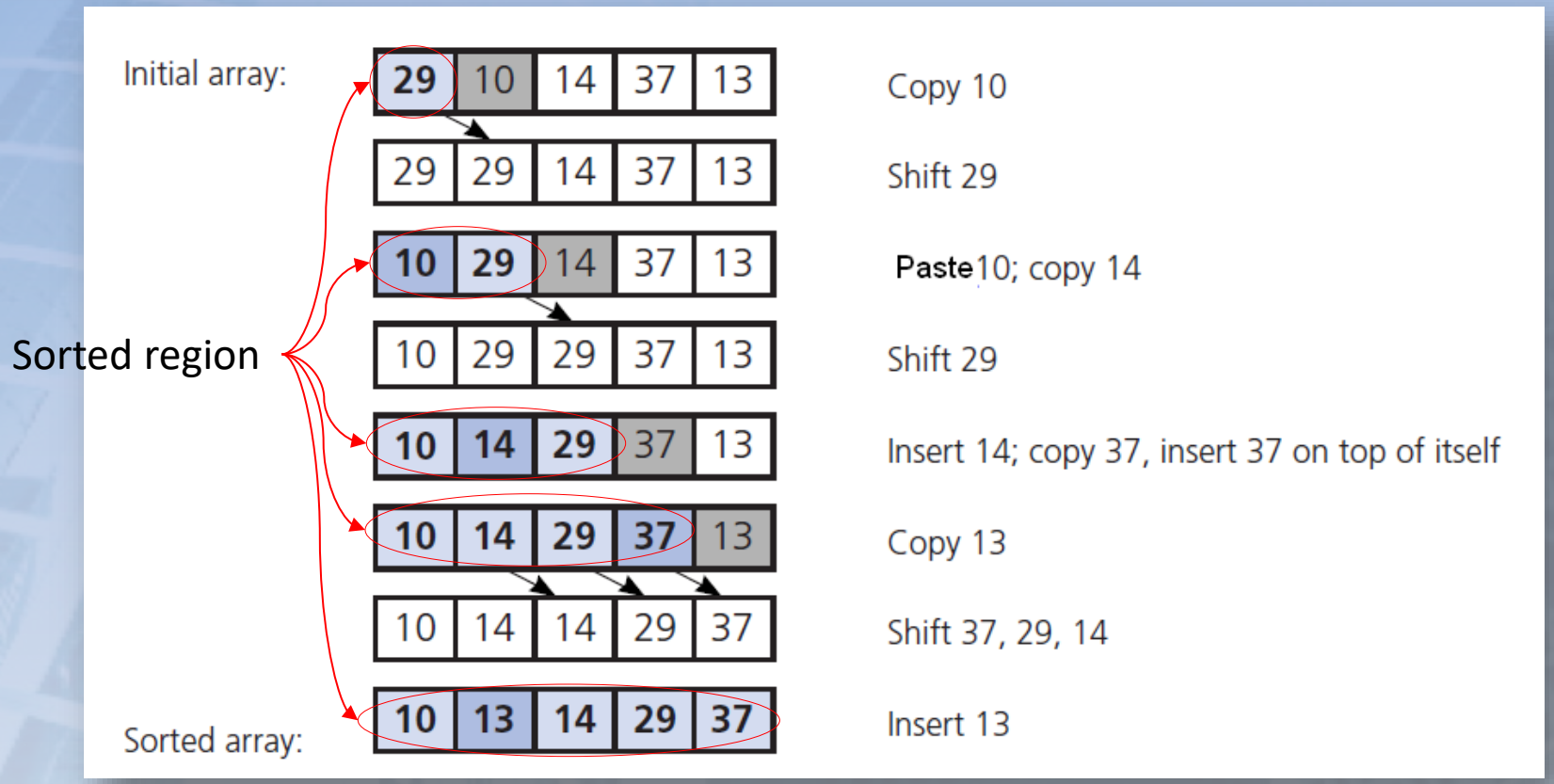


FIGURE 11-4 An insertion sort of an array of five integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

```
/** Sorts the items in an array into ascending order.
 @pre  None.
 @post  theArray is sorted into ascending order; n is unchanged.
 @param theArray  The given array.
 @param n  The size of theArray. */
template<class ItemType>
void insertionSort(ItemType theArray[], int n)
{
   // unsorted = first index of the unsorted region,
   // loc = index of insertion in the sorted region,
   // nextItem = next item in the unsorted region.
   // Initially,  sorted region is theArray[0],
   //             unsorted region is theArray[1..n-1].
   // In general, sorted region is theArray[0..unsorted-1],
   //             unsorted region theArray[unsorted..n-1]
```

LISTING 11-3 An implementation of the insertion sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

```
    for (int unsorted = 1; unsorted < n; unsorted++)
    {
        // At this point, theArray[0..unsorted-1] is sorted.
        // Find the right position (loc) in theArray[0..unsorted]
        // for theArray[unsorted], which is the first entry in the
        // unsorted region; shift, if necessary, to make room
        ItemType nextItem = theArray[unsorted];
        int loc = unsorted;
        while ((loc > 0) && (theArray[loc - 1] > nextItem))
        {
            // Shift theArray[loc - 1] to the right
            theArray[loc] = theArray[loc - 1];
            loc--;
        }  // end while
        // At this point, theArray[loc] is where nextItem belongs
        theArray[loc] = nextItem; // Insert nextItem into sorted region
    }  // end for
}  // end insertionSort
```

Note this is a nested loop for n

LISTING 11-3 An implementation of the insertion sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

- Analysis
  - Worst case $O(n^2)$
  - Best case (array already in order) is $O(n)$
- Unsuitable for large arrays
  - Unless already sorted
- Pros
  - Easy to understand
  - No additional space required
  - Preserves relative order of items with equal keys (stable)
- Cons
  - $O(n^2)$ – slow
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Faster Sorting Algorithms

- The Merge Sort
- The Quick Sort
- The Radix Sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

- Divide and conquer sorting algorithm
- Halve the array, sort its halves, and then merge the halves

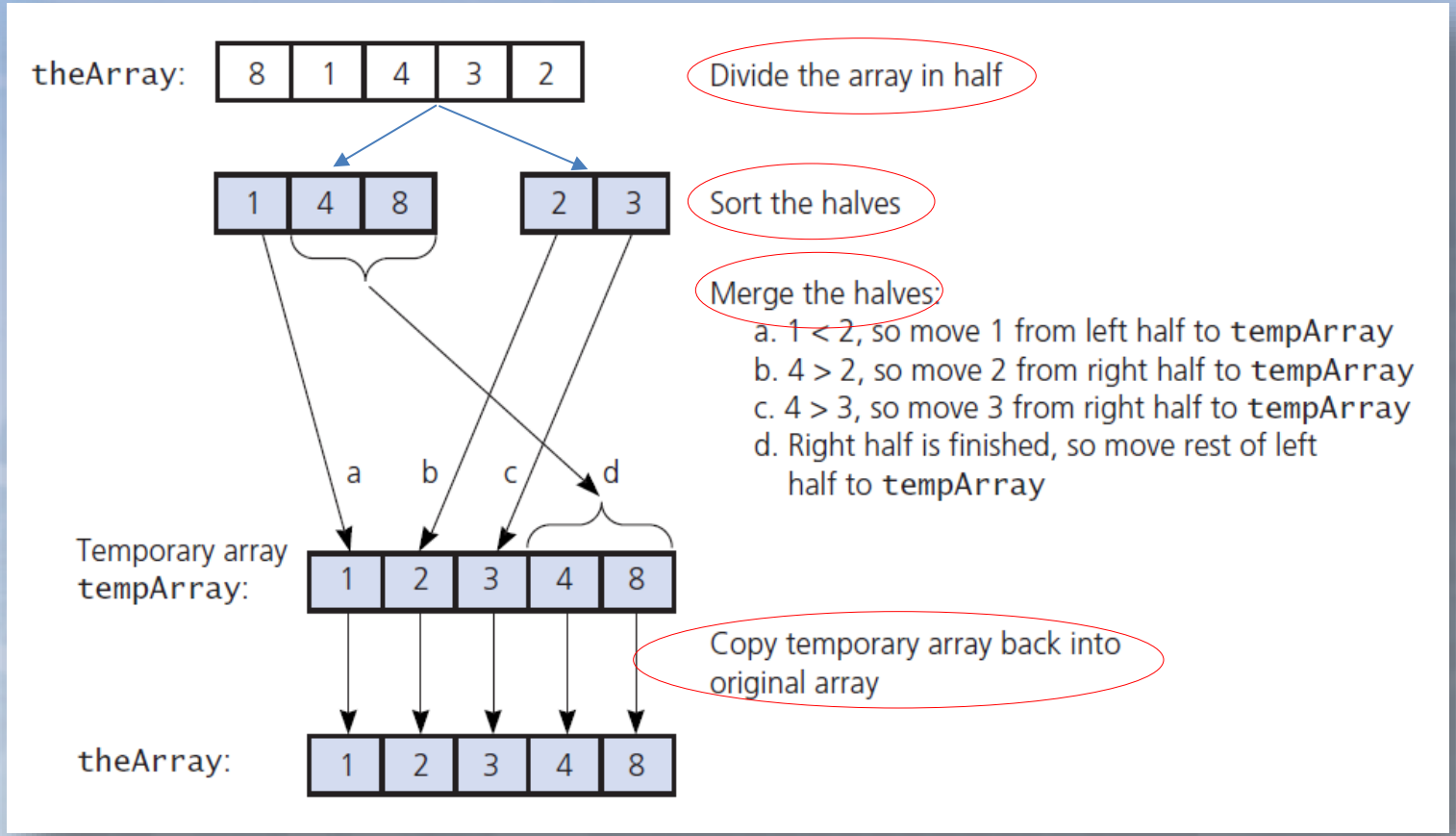


FIGURE 11-5 A merge sort with an auxiliary temporary array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

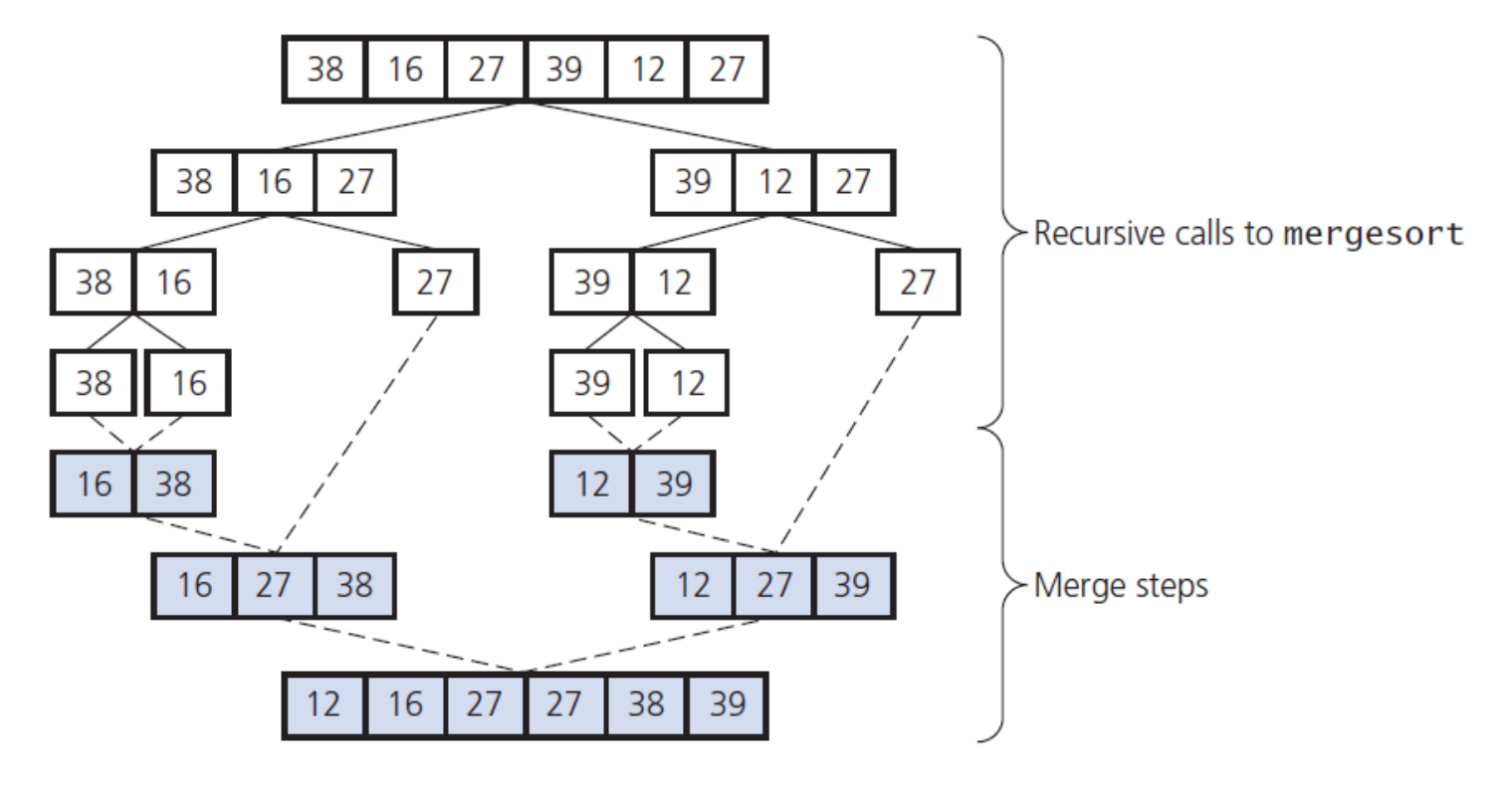


FIGURE 11-6 A merge sort of an array of six integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

```
// Sorts theArray[first..last] by
//    1. Sorting the first half of the array
//    2. Sorting the second half of the array
//    3. Merging the two sorted halves
mergeSort(theArray: ItemArray, first: integer, last: integer)
{
   if (first < last)
   {
      mid = (first + last) / 2          // Get midpoint

      // Sort theArray[first..mid]
      mergeSort(theArray, first, mid)

      // Sort theArray[mid+1..last]
      mergeSort(theArray, mid + 1, last)

      // Merge sorted halves theArray[first..mid] and theArray[mid+1..last]
      merge(theArray, first, mid, last)
   }
   // If first >= last, there is nothing to do
}
```

Pseudocode for the merge sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

- Analysis
  - O(n log n)
  - Worse case O(n log n)
- Pros
  - Performs well even on large number of items
  - Simple to implement
  - Preserves relative order of items with equal keys (stable)
- Cons
  - Requires additional memory O(n)
  - Requires copying of entries

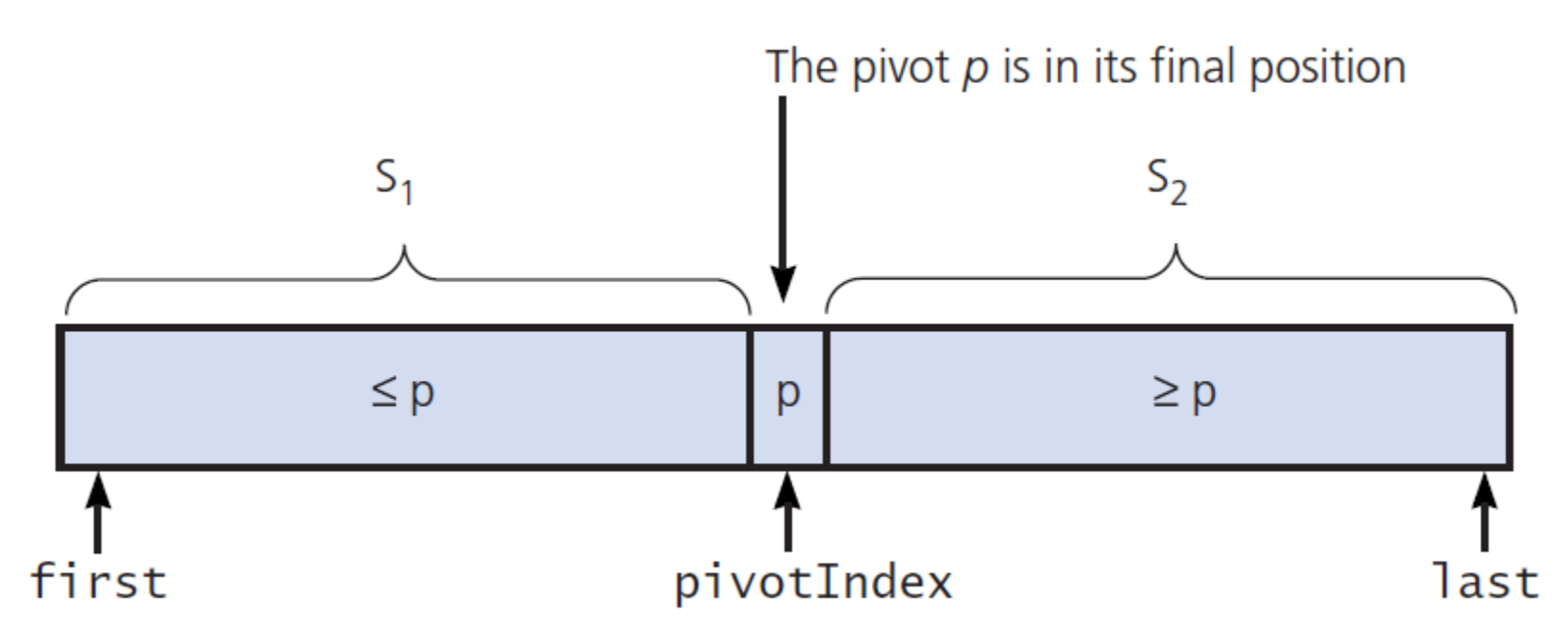
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Another divide-and-conquer algorithm
  - Choose a pivot
  - Partition an array into items that are
    - Less than or equal to the pivot and
    - Those that are greater than or equal to the pivot
  - Recursively apply the same process to the partitioned sub-arrays
  - Recursion stops when there is one element remaining in the sub-array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort



Quick sort algorithm

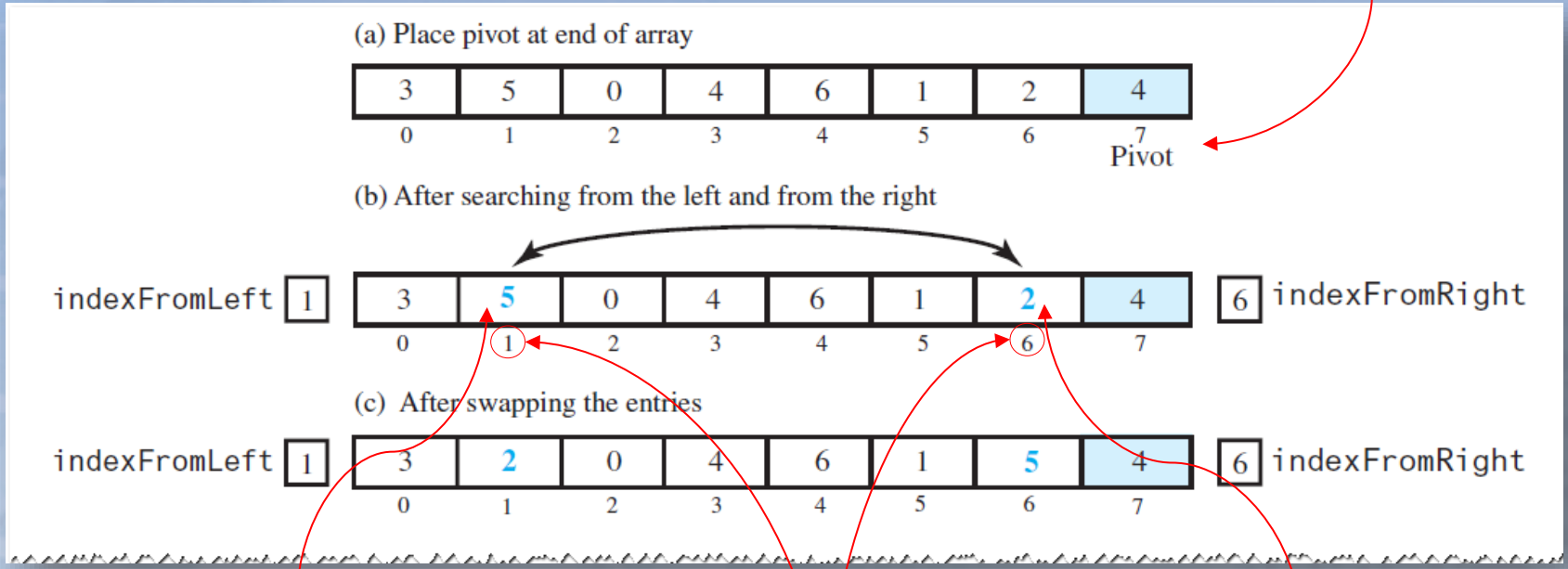
1. Choose a pivot
2. Partition the array
3. Recursively call

FIGURE 11-9 A partition about a pivot

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

1. Choose a pivot



2. Start left to right, find 1st entry that is >= pivot

3. Start right to left, find 1st entry that is <= pivot

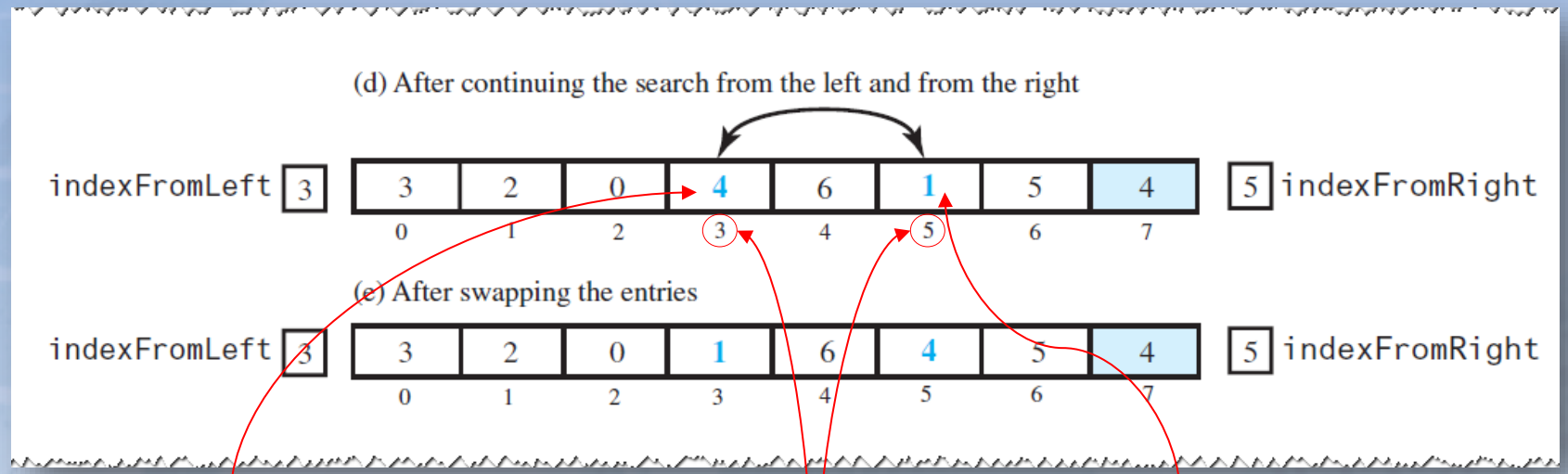
4. if indexFromLeft is less than indexFromRight, swap the two entries, else search ends

5. Continue left and right searches

FIGURE 11-10 A partitioning of an array during a quick sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort



6. Find next entry that is >= pivot

7. Find next entry that is <= pivot

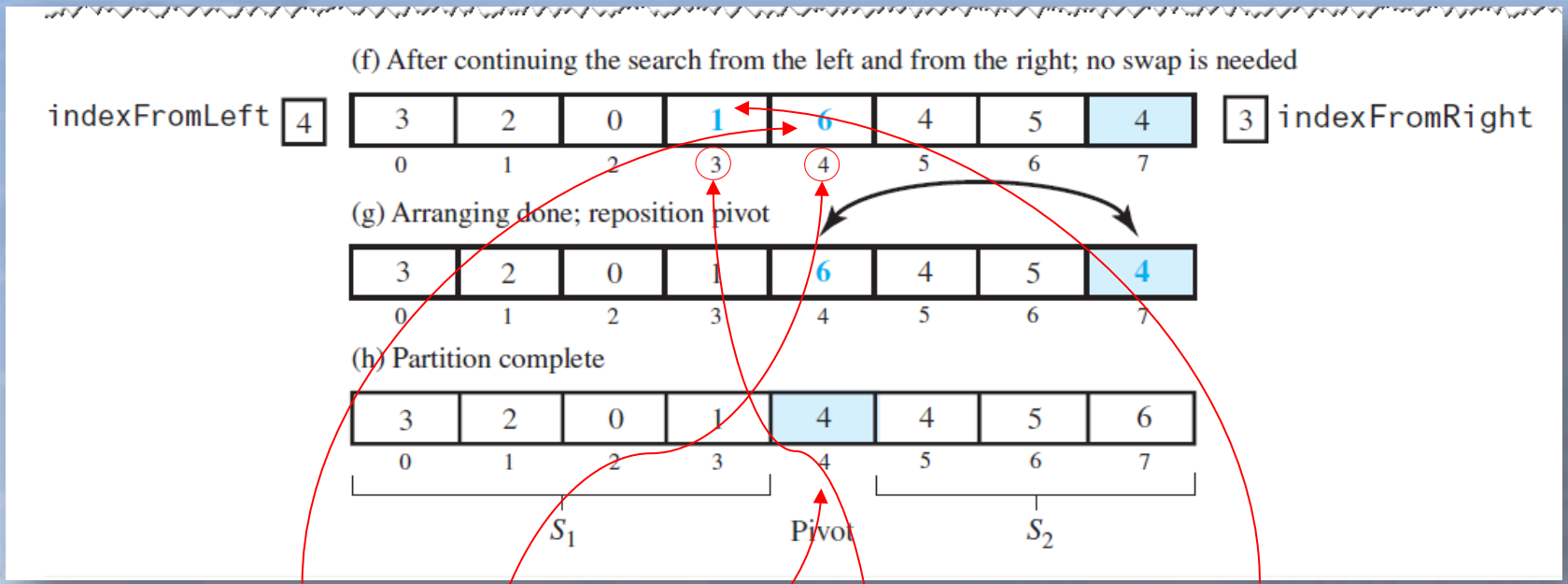
8. if indexFromLeft is less than indexFromRight, swap the two entries, else search ends

9. Continue left and right searches

FIGURE 11-10 A partitioning of an array during a quick sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort



10. Find next entry that is >= pivot

11. Find next entry that is <= pivot

12. if indexFromLeft is less than indexFromRight, swap the two entries, else **search ends**

13. place the pivot between the subarrays S1 and S2 by swapping a[indexFromLeft] and a[last]; partition complete

14. Recursively call

FIGURE 11-10 A partitioning of an array during a quick sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Pivot selection goal
  - Subarrays each have the same or nearly the same number of entries
- Pivot selection strategy
  - Last entry
  - First entry
  - Middle entry
  - Median-of-three

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Analysis
  - Partitioning is an O(n) task
  - There are either $\log_2 n$ or 1 + $\log_2 n$ levels of recursive calls to quickSort.
- We conclude
  - Worst case O(n^2)
  - Average case O(n log n)
- Pros
  - Divide and conquer algorithm that is efficient on large number of items
  - No additional memory required
- Cons
  - Worst case O(n^2)
  - Does not preserve relative order of items with equal keys (not stable)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

- Different from other sorts
  - Does not compare entries in an array
- Begins by organizing data (say strings) according to least significant letters
  - Then combine the groups
- Next form groups using next least significant letter

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

0123, 2154, 0222, 0004, 0283, 1560, 1061, 2150	Original integers
(1560, 2150) (1061) (0222) (0123, 0283) (2154, 0004)	Grouped by fourth digit
1560, 2150, 1061, 0222, 0123, 0283, 2154, 0004	Combined
(0004) (0222, 0123) (2150, 2154) (1560, 1061) (0283)	Grouped by third digit
0004, 0222, 0123, 2150, 2154, 1560, 1061, 0283	Combined
(0004, 1061) (0123, 2150, 2154) (0222, 0283) (1560)	Grouped by second digit
0004, 1061, 0123, 2150, 2154, 0222, 0283, 1560	Combined
(0004, 0123, 0222, 0283) (1061, 1560) (2150, 2154)	Grouped by first digit
0004, 0123, 0222, 0283, 1061, 1560, 2150, 2154	Combined (sorted)

FIGURE 11-13 A radix sort of eight integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

- Analysis
  - Requires *n* moves each time it forms groups
  - *n* moves to combine again into one group
  - Performs these $2 \times n$ moves *d* times
  - Thus requires $2 \times n \times d$ moves
  - More appropriate for chain of linked lists than for an array
- Radix sort is of order O(*n*)
- Pros
  - Radix sort is of order O(n)
  - Often faster than merge sort or quick sort for large number of items
  - Preserves relative order of items with equal keys (stable)
- Cons
  - Time complexity grows linearly with the number of digits
  - Requires same size keys, with standard way of breaking the keys into pieces

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Comparison of Sorting Algorithms

	Worst case	Average case
Selection sort	n^2	n^2
Bubble sort	n^2	n^2
Insertion sort	n^2	n^2
Merge sort	$n \times \log n$	$n \times \log n$
Quick sort	n^2	$n \times \log n$
Radix sort	n	n
Tree sort	n^2	$n \times \log n$
Heap sort	$n \times \log n$	$n \times \log n$

FIGURE 11-14 Approximate growth rates of time required for eight sorting algorithms

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Representation

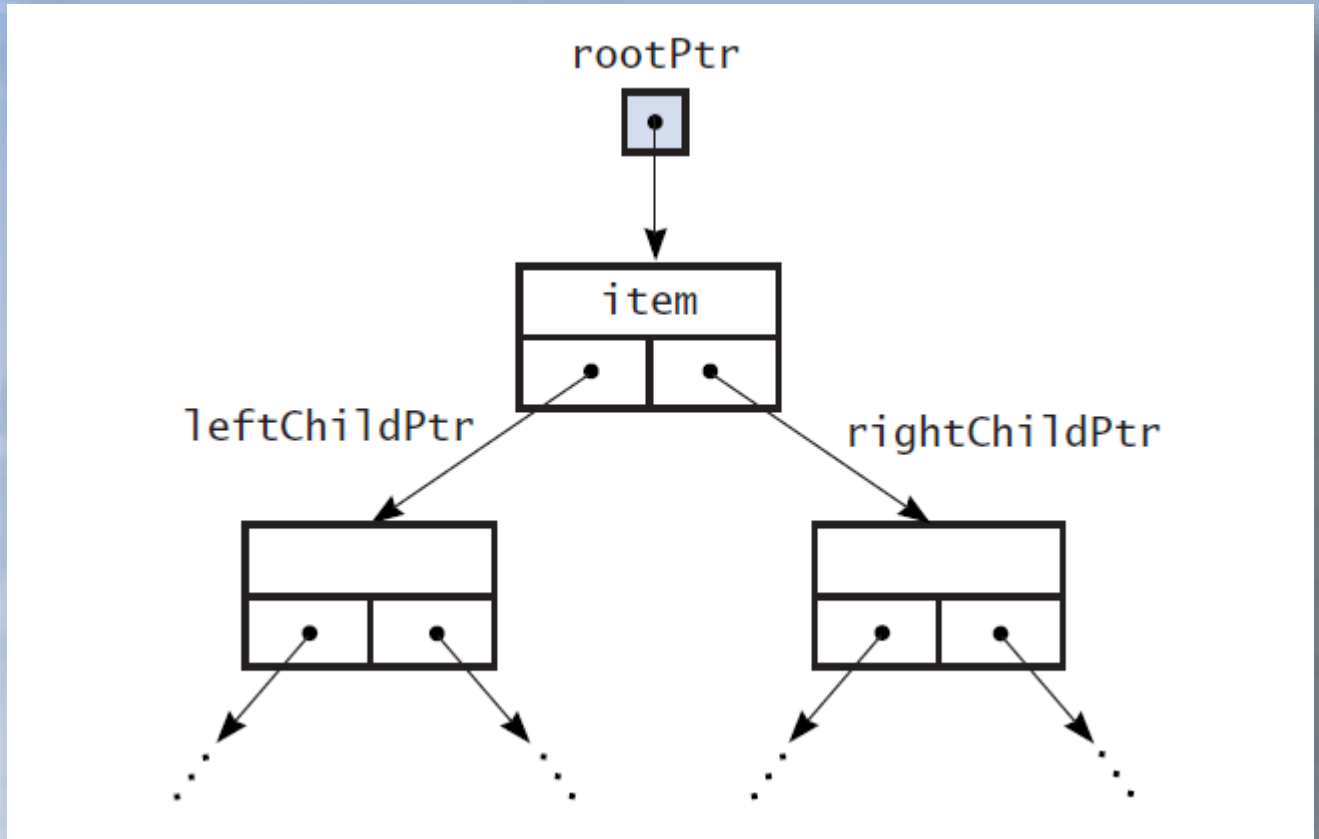


Figure 16-2 Most common way of implementing a tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Header File

```
//-----------------------------------------------------------
//       Public BinaryTreeInterface Methods Section.
//-----------------------------------------------------------
   bool isEmpty() const;
   int getHeight() const;
   int getNumberOfNodes() const;
   ItemType getRootData() const throw(PrecondViolatedExcept);
   void setRootData(const ItemType& newData);
   bool add(const ItemType& newData); // Adds an item to the tree
   bool remove(const ItemType& data); // Removes specified item from the tree
   void clear();
   ItemType getEntry(const ItemType& anEntry) const throw(NotFoundException);
   bool contains(const ItemType& anEntry) const;

//-----------------------------------------------------------
//       Public Traversals Section.
//-----------------------------------------------------------
   void preorderTraverse(void visit(ItemType&)) const;
   void inorderTraverse(void visit(ItemType&)) const;
   void postorderTraverse(void visit(ItemType&)) const;

//-----------------------------------------------------------
//       Overloaded Operator Section.
//-----------------------------------------------------------
   BinaryNodeTree& operator=(const BinaryNodeTree& rightHandSide);
}; // end BinaryNodeTree

#include "BinaryNodeTree.cpp"
#endif
```

Public traversal methods each call a protected method that performs the actual recursion

LISTING 16-3 A header file for the link-based implementation of the class BinaryNodeTree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

The constructors. The public constructors have the following definitions in the implementation file:

```
template<class ItemType>
BinaryNodeTree<ItemType>::BinaryNodeTree() : rootPtr(nullptr)
{
} // end default constructor

template<class ItemType>
BinaryNodeTree<ItemType>::BinaryNodeTree(const ItemType& rootItem)
{
   rootPtr = new BinaryNode<ItemType>(rootItem, nullptr, nullptr);
}  // end constructor

template<class ItemType>
BinaryNodeTree<ItemType>::BinaryNodeTree(const ItemType& rootItem,
                          const BinaryNodeTree<ItemType>* leftTreePtr,
                          const BinaryNodeTree<ItemType>* rightTreePtr)
{
   rootPtr = new BinaryNode<ItemType>(rootItem,
                                      copyTree(leftTreePtr->rootPtr),
                                      copyTree(rightTreePtr->rootPtr));
}  // end constructor
```

set the value of rootPtr to nullptr

sets rootPtr to point to a new node containing the given data item

sets rootPtr to point to a new node containing the given data item and pointers to copies of the given subtrees

Constructors

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
template<class ItemType>
std::shared_ptr<BinaryNode<ItemType>> BinaryNodeTree<ItemType>::copyTree(
     const std::shared_ptr<BinaryNode<ItemType>> oldTreeRootPtr) const
{
   std::shared_ptr<BinaryNode<ItemType>> newTreePtr;

   // Copy tree nodes during a preorder traversal
   if (oldTreeRootPtr != nullptr)
   {
      // Copy node
      newTreePtr = std::make_shared<BinaryNode<ItemType>>(oldTreeRootPtr->getItem(),
                                                          nullptr, nullptr);
      newTreePtr->setLeftChildPtr(copyTree(oldTreeRootPtr->getLeftChildPtr()));
      newTreePtr->setRightChildPtr(copyTree(oldTreeRootPtr->getRightChildPtr()));
   }  // end if
   // Else tree is empty (newTreePtr is nullptr)

   return newTreePtr;
}  // end copyTree
```

To copy a tree, traverse it in **preorder** and insert each item visited into a new node

Protected method copyTree called by copy constructor

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

To delete a tree, traverse it in **postorder** and delete

```
template<class ItemType>
void BinaryNodeTree<ItemType>::
     destroyTree(std::shared_ptr<BinaryNode<ItemType>> subTreePtr)
{
   if (subTreePtr != nullptr)
   {
      destroyTree(subTreePtr->getLeftChildPtr());
      destroyTree(subTreePtr->getRightChildPtr());
      subTreePtr.reset(); // Decrement reference count to node
   }  // end if
}  // end destroyTree
```

destroyTree used by destructor which simply calls this method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

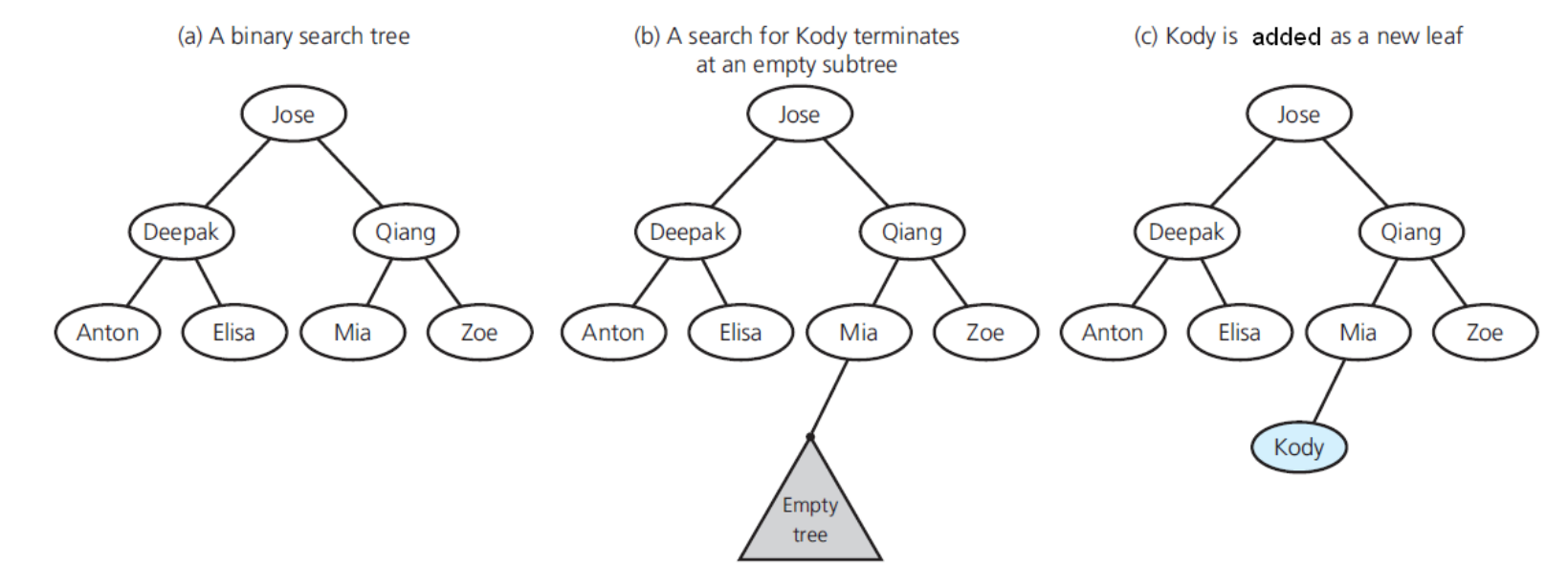
The Implementation

```
template<class ItemType>
int BinaryNodeTree<ItemType>::
    getHeightHelper(std::shared_ptr<BinaryNode<ItemType>> subTreePtr) const
{
   if (subTreePtr == nullptr)
      return 0;
   else
      return 1 + max(getHeightHelper(subTreePtr->getLeftChildPtr()),
                     getHeightHelper(subTreePtr->getRightChildPtr()));
}  // end getHeightHelper
```

Protected method getHeightHelper

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree



- If you take the output of a preorder traversal of a binary search tree and use it with insertInorder to create a binary search tree, you will obtain a duplicate tree. Tree (a) was used in previous chapter.
- The copy constructor for the ADT binary tree used a preorder traversal to copy the tree.
- To insert Kody, search the BST, and search algorithm would terminate at Mia's left child

FIGURE 16-7 Adding Kody to a binary search tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

3 cases for removing node *N*:

1. *N* is a leaf
   - Remove leaf containing target
   - Set pointer in parent to nullptr
2. *N* has only left (or right) child – cases are symmetrical
   - After *N* removed, all data items rooted at *L* (or *R*) are adopted by root of *N*
   - All items adopted are in correct order, binary search tree property preserved
3. *N* has two children
   - Locate another node *M* easier to remove from tree than *N*
   - Copy item that is in *M* to *N*
   - Remove *M* from tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

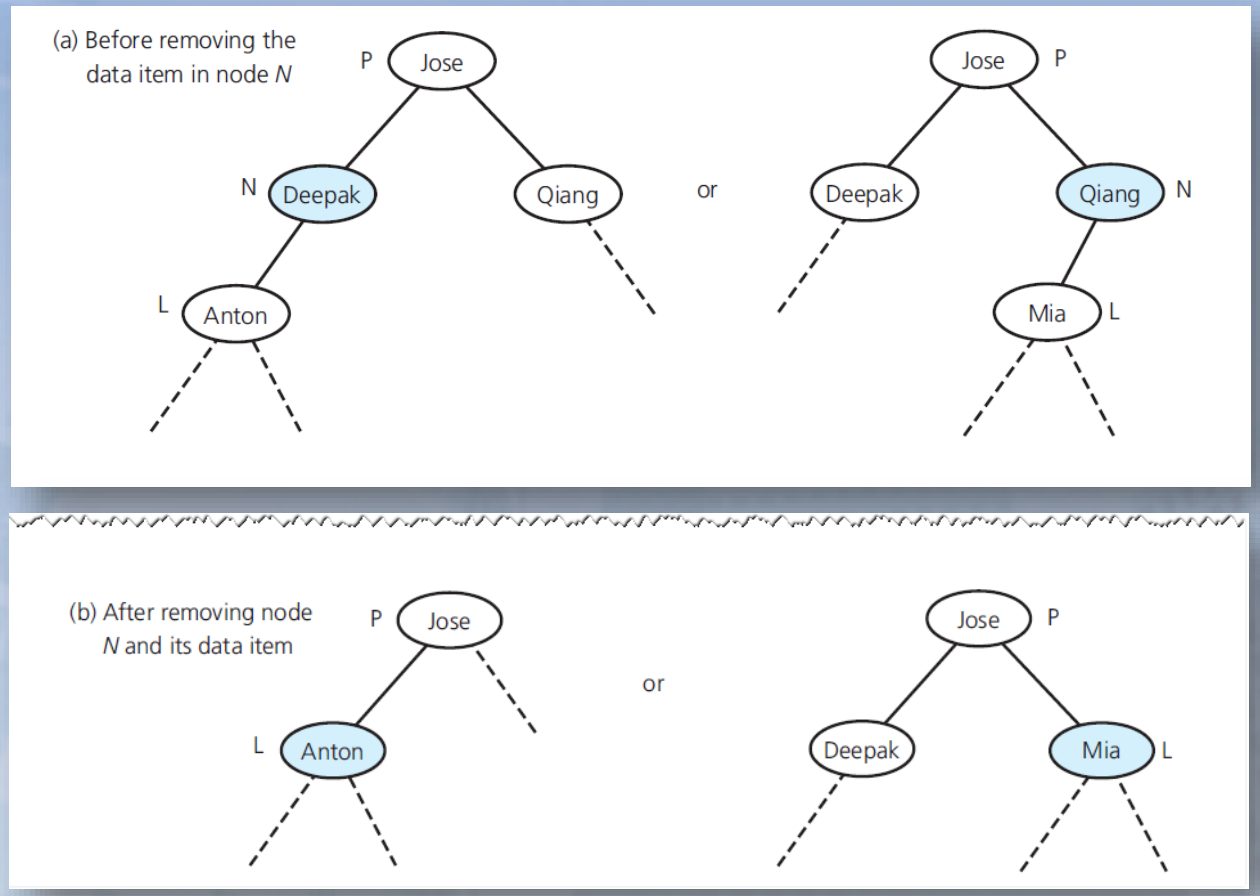


Figure 16-9 Case 2 for removeValue: The data item to remove is in a node N that has only a left child and whose parent is node P

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

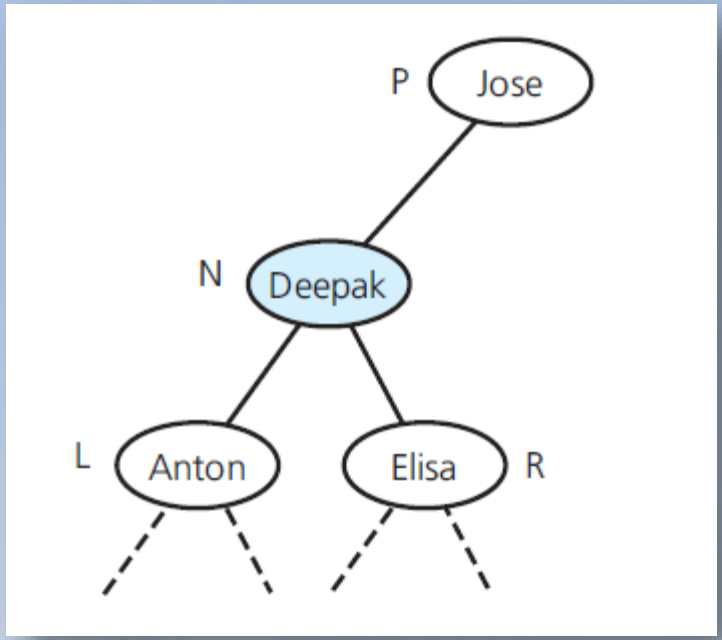


FIGURE 16-10 Case 3: The data item to remove is in a node *N* that has two children

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

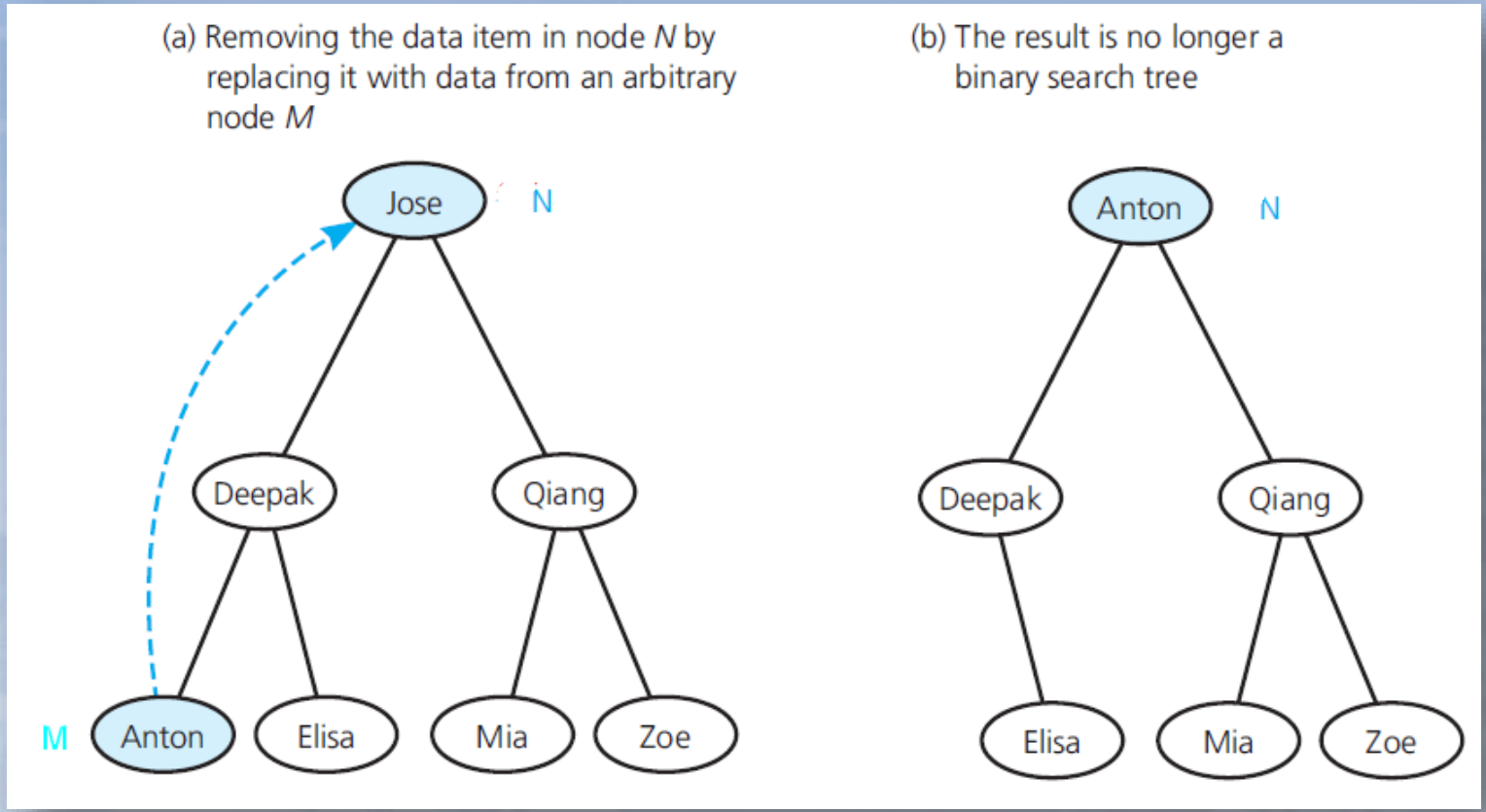
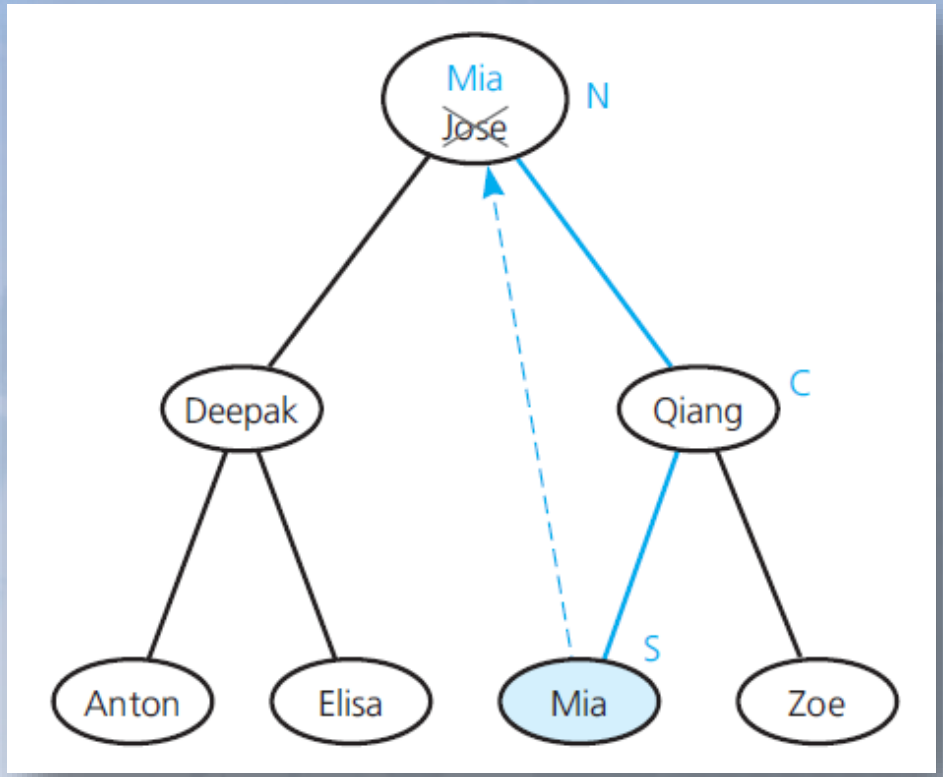


FIGURE16-11 Not any node will do

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

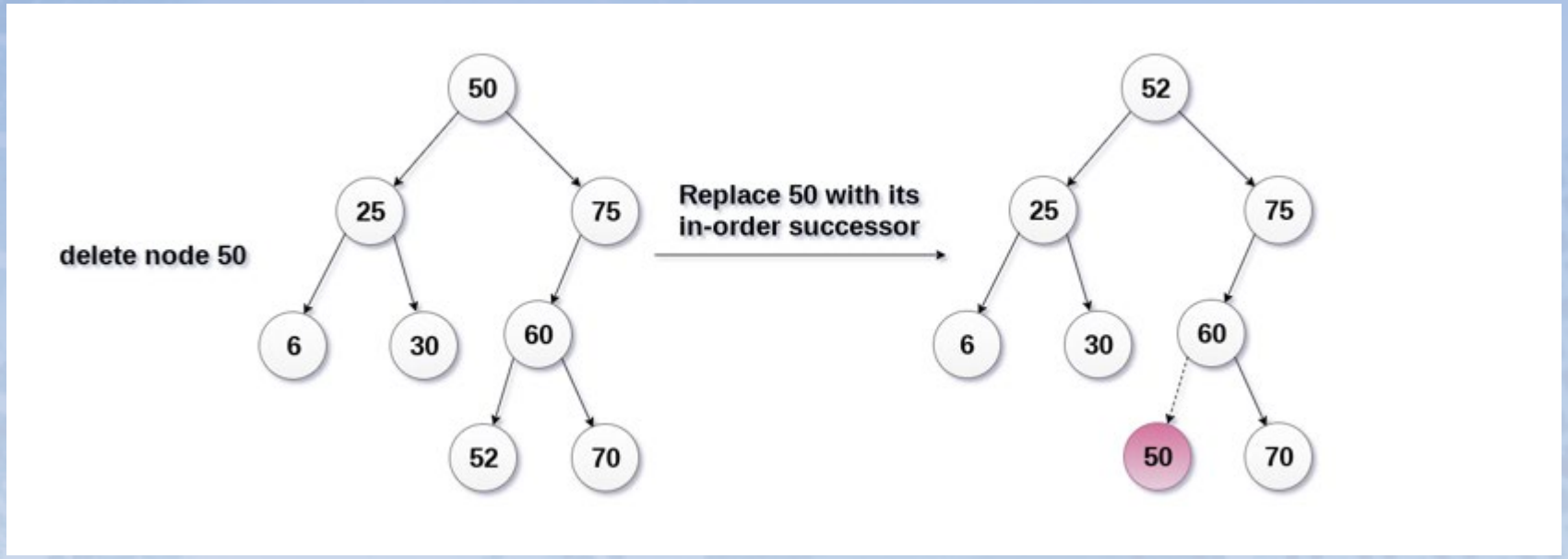


- inorder successor of a data item is the leftmost node of N's right subtree.

FIGURE 16-13 Replacing the data item in node *N* with its **inorder successor**

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree



https://www.javatpoint.com/deletion-in-binary-search-tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Saving a Binary Search Tree in a File

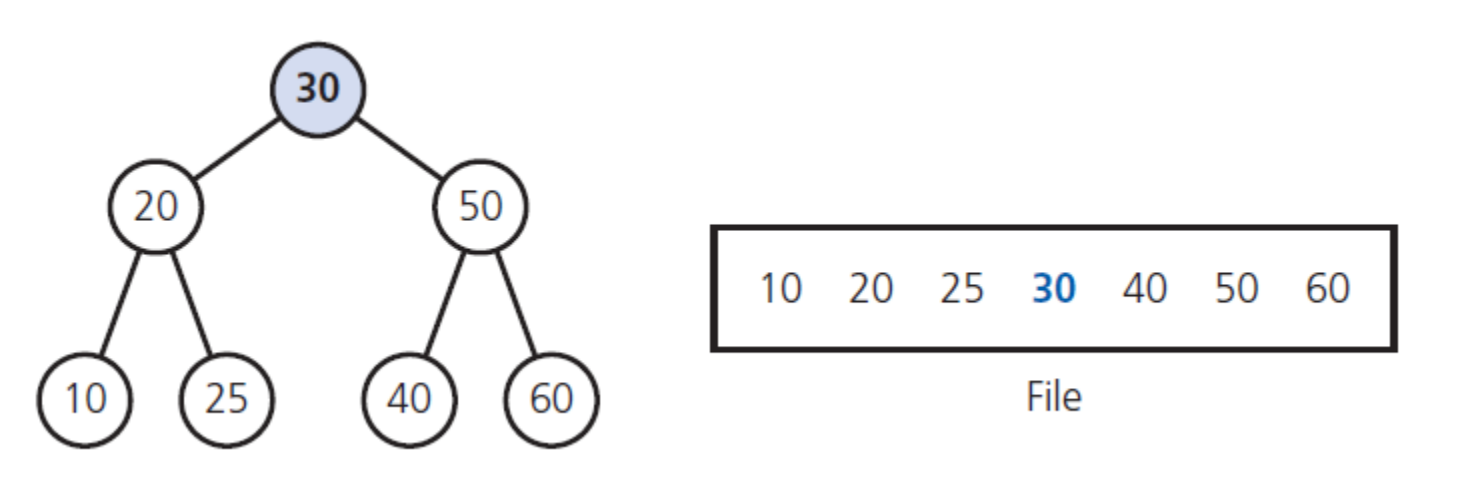


FIGURE 16-16 A full tree saved in a file by using inorder traversal

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Tree Sort

```
// Sorts the integers in an array into ascending order.
treeSort(anArray: array, n: integer)
{
    Add anArray's entries to a binary search tree bst
    Traverse bst in inorder. As you visit bst's nodes, copy their data items into successive
     locations of anArray
}
```

- Tree sort:
  - Insertion into BST is O(log n) in the average case, and O(n) in the worse case
  - Thus, insertion into BST is O(n × log n) in the average case, and $O(n^2)$ in the worst case
  - Traversal involves one copy: O(n)
  - Thus, tree sort is O(n × log n) in the average case, and $O(n^2)$ in the worst case

Tree sort uses a binary search tree.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

General Trees

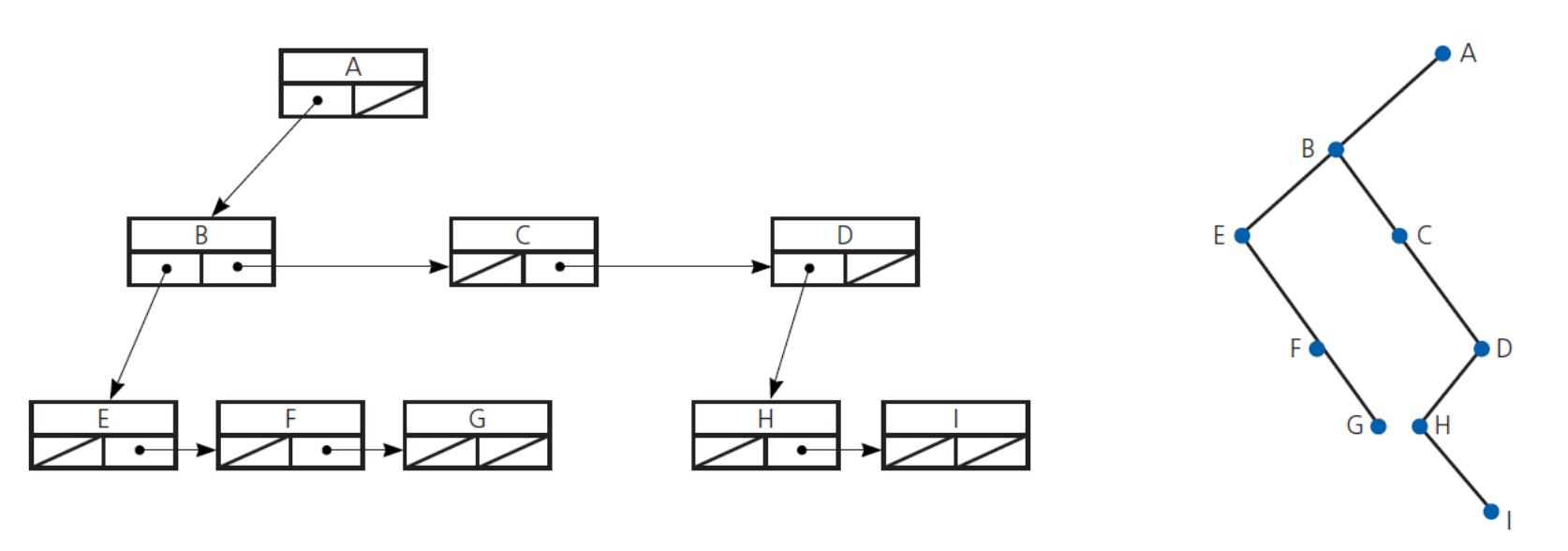
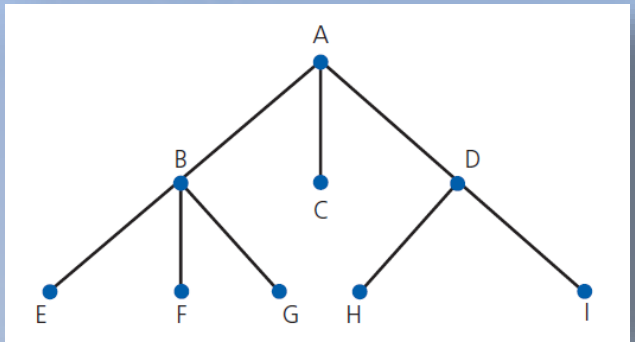


FIGURE 16-19 An implementation of a general tree and its equivalent binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

General Trees



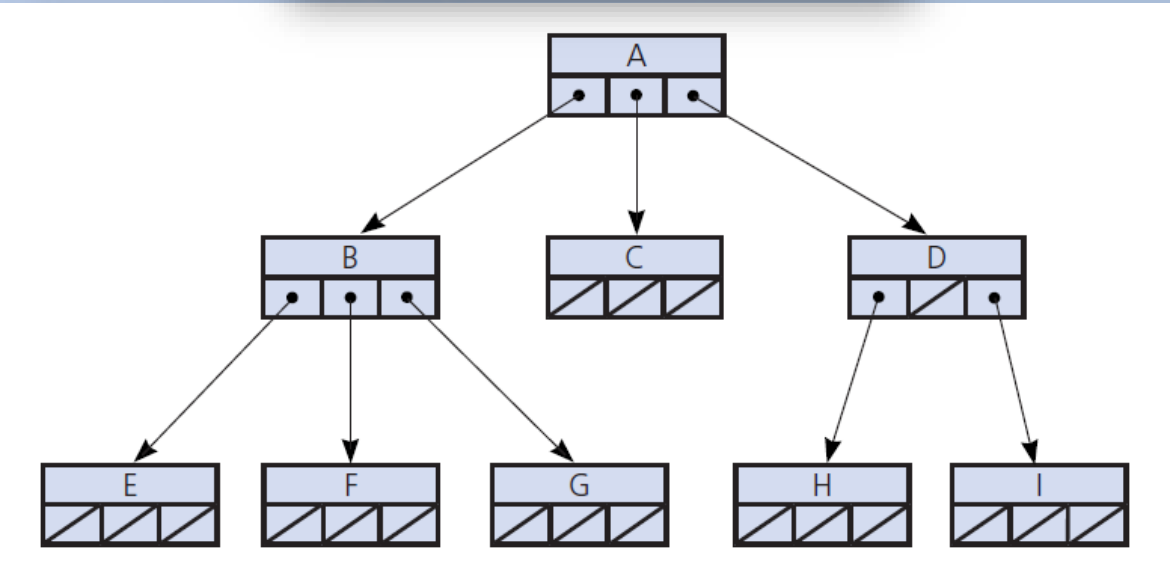


FIGURE 16-20 An implementation of the *n*-ary tree in Figure 16-18

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End